CONFIDENTIAL · SECURITY AUDIT

VERSION 3 · RE-REVIEW

Security Audit Tech for Palestine Website

techforpalestine.org · Cloudflare Pages & Workers

This is the **third-round re-review**, assessing patches applied after v2 (2026-04-02). Ten of the seventeen prior findings have now been resolved. This report documents two new medium-severity issues introduced by the latest changes and tracks all remaining open findings.

ORIGINAL AUDIT	V2 RE-REVIEW	V3 RE-REVIEW
2026-03-31 (v1)	2026-04-02	2026-04-06

Executive Summary — v3 Re-review

Delta audit of patches applied after v2 report, plus all remaining open findings.

10	0	2	5	3
FIXED	CRITICAL	HIGH	MEDIUM	LOW

Significant progress: all three v1 critical findings and all three new v2 findings are now resolved. No critical issues remain. The two outstanding high-severity issues — a payload echo exposing donor PII and the absence of rate limiting — should be the immediate focus. Two new medium-severity issues were introduced: `'unsafe-inline'` in the Content Security Policy (which largely defeats XSS protection), and a residual client-side secret embedded in the Axios bundle.

FULL ISSUE REGISTER — V3 STATUS

ID	Finding	Severity	v3 Status
C-1	API key & account ID hardcoded in wrangler configs	CRITICAL	Fixed
C-2	Unauthenticated membership invite endpoint	CRITICAL	Fixed
C-3	Unverified webhook triggers privileged invite	CRITICAL	Fixed
N-1	Webhook secret in URL query parameter	CRITICAL	Fixed
N-2	Timing attack on token comparison	HIGH	Fixed
N-3	membership-invite proxies arbitrary body	MEDIUM	Fixed
H-1	Secret key logged to browser console	HIGH	Fixed (console.log removed — residual: see R-1)
H-3	Middleware caches POST/API responses as public	HIGH	Fixed
H-5	Wildcard CORS on all endpoints	HIGH	Fixed
M-3	getEnv globalThis fallback	MEDIUM	Fixed
M-5	Internal error.message in HTTP responses	MEDIUM	Fixed

ID	Finding	Severity	v3 Status
M-6	Missing security headers	MEDIUM	Fixed (partial — see R-2)
L-1	Notion DB IDs in wrangler.toml	LOW	Fixed
R-1	Residual: PUBLIC_SECRET_KEY still in client-side Axios bundle	MEDIUM	New
R-2	CSP uses 'unsafe-inline' — defeats XSS script protection	MEDIUM	New
H-2	Full donor PII payload echoed in HTTP response	HIGH	Open
H-4	No rate limiting on any endpoint	HIGH	Open
M-1	No CSRF / Origin check on form endpoints	MEDIUM	Open
M-2	No input format/length validation on form endpoints	MEDIUM	Open
M-4	Donor email addresses logged in webhook (PII in logs)	MEDIUM	Partial
L-2	QGiv embed script loaded without SRI hash	LOW	Open
L-3	Image proxy has no authentication	LOW	Open

CONFIRMED FIXED SINCE V2

- **N-1 — Webhook secret moved to `X-Webhook-Secret` header** — `qgiv-webhook.ts:18` now reads `request.headers.get("X-Webhook-Secret")`. No longer present in URLs or logs.
- **N-2 — Constant-time comparison implemented** — new `src/utils/crypto.ts` exports `constantTimeEqual()` using `timingSafeEqual` from `node:crypto` via `TextEncoder`. Used in both endpoints.
- **N-3 — membership-invite body validated and whitelisted** — validates email format with regex, restricts `type` to `"paid"` or `"exemption"`, validates optional `paymentReference`. Only whitelisted fields forwarded to Hub API.
- **H-1 — `console.log("PUBLIC_SECRET_KEY:", ...)` removed** from `api.ts`. The key no longer prints to the browser console. (A residual client-side exposure remains — see R-1.)
- **H-3 — Middleware cache fix applied** — now checks `isApi` and `isGet`; sets `no-store` for API and POST responses, `public, max-age=600` only for GET page responses.
- **H-5 — CORS restricted** on `endorsement-request.ts`, `e4p-pledge-sign.ts`, and the webhook OPTIONS handler to `https://techforpalestine.org`. No more wildcard on write endpoints.
- **M-3 — `globalThis` property fallback removed** from `getEnv.ts`. Now resolves only from `locals.runtime.env`, `import.meta.env`, and `process.env`.
- **M-5 — Error details removed from HTTP responses** — `qgiv-webhook.ts` no longer returns `error.message` or Plausible exception details in response bodies.
- **M-6 — Security headers added** via `public/_headers` — includes `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, and a `Content-Security-Policy`. (CSP has an issue — see R-2.)
- **L-1 — Notion DB IDs removed from `wrangler.toml`** — comment now directs to Cloudflare Pages dashboard / `.env` file.

NEW FINDINGS INTRODUCED BY V3 PATCH

NEW MEDIUM R-1

PUBLIC_SECRET_KEY Still Embedded in Client-Side Axios Bundle

File: src/store/api.ts:4–13 **Page:** /project-apply (client-side React bundle)

OWASP: A02:2021 – Cryptographic Failures

The `console.log` calls have been correctly removed. However, `PUBLIC_SECRET_KEY` is still loaded at module level via `getEnv("PUBLIC_SECRET_KEY")` and placed directly into the Axios instance `Authorization` header that ships in the client-side JavaScript bundle:

```
// api.ts:4-13 — runs in every visitor's browser on /project-apply
const SECRET_KEY = getEnv("PUBLIC_SECRET_KEY");

const axiosInstance = axios.create({
  baseURL: API_URL,
  headers: {
    "Content-Type": "application/json",
    Authorization: SECRET_KEY,  // ← bundled into browser JS
  },
});
```

Any visitor to `/project-apply` can open DevTools → Network tab, submit the form, and read the `Authorization` header value. The key is also embedded in the compiled JavaScript bundle and can be extracted by anyone who downloads and searches it.

SECURE FIX

Create a server-side Astro API route (e.g. `/api/project-form`) that holds the credential server-side and proxies the request to the external API. The browser submits to `/api/project-form` with no auth header; the server-side route adds the credential before calling upstream. This removes the secret from the browser entirely.

NEW MEDIUM R-2

Content Security Policy Uses 'unsafe-inline' — XSS Protection Defeated

File: public/_headers:6 **OWASP:** A05:2021 – Security Misconfiguration

The newly added CSP includes `'unsafe-inline'` in both `script-src` and `style-src`. For `script-src` in particular, this completely negates the primary XSS benefit of having a CSP: any injected inline `<script>` tag will be permitted to execute.

```
# public/_headers — current (vulnerable)
Content-Security-Policy: default-src 'self';
  script-src 'self' 'unsafe-inline' https://secure.qgiv.com ...;
  style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
```

`'unsafe-inline'` is typically added because frameworks inject inline scripts or styles. The correct approach is to use a CSP nonce or hash instead of a blanket `'unsafe-inline'` allowance.

SECURE FIX — USE NONCES (PREFERRED) OR STRICT-DYNAMIC

```
# Option A — strict-dynamic (allows framework-generated scripts, blocks injected on
es)
script-src 'self' 'strict-dynamic' 'nonce-{RANDOM}' https://secure.qgiv.com;

# Option B — hash-based (for known static inline scripts)
script-src 'self' 'sha256-{HASH_OF_INLINE_SCRIPT}' https://secure.qgiv.com;

# For styles, unsafe-inline is lower risk but can still be replaced:
style-src 'self' https://fonts.googleapis.com 'sha256-{HASH}';
```

Astro's Cloudflare adapter supports nonce injection via middleware. Alternatively, move all inline styles to external stylesheets and eliminate the need for `'unsafe-inline'` entirely.

OPEN FINDINGS — HIGH SEVERITY

HIGH H-2 Full Donor PII Payload Echoed Back in HTTP Response Body

File: src/pages/api/qgiv-webhook.ts:184–196 **OWASP:** A02:2021 – Cryptographic Failures

When the donation type cannot be determined from the payload, the entire raw QGiv webhook payload — which contains donor name, email address, postal address, donation amount, and transaction ID — is returned verbatim in the HTTP response body. This has been open since v1.

```
// qgiv-webhook.ts:184-196 — unchanged across all three reviews
return new Response(JSON.stringify({
  success: true,
  message: "Webhook received but donation type unclear",
  payload,  // ← full unredacted donor record returned to caller
}), ...);
```

Although the endpoint now requires a valid `X-Webhook-Secret` header, the authenticated caller (QGiv's servers) receives this PII reflection on every indeterminate payload. More importantly, if the secret is ever compromised, a forged request that doesn't match the form/recurring logic will leak donor PII directly in the response.

FIX — ONE LINE CHANGE

```
// Replace the entire fallthrough return with:
return new Response(
  JSON.stringify({ success: true, message: "Webhook received but donation type uncl
ear" }),
  { status: 200, headers: { "Content-Type": "application/json" } }
);
```

HIGH H-4

No Rate Limiting on Any Endpoint

File: All src/pages/api/* endpoints **OWASP:** A04:2021 – Insecure Design

Every write endpoint — including those that create Notion records, trigger Hub invitations, and subscribe email addresses — has zero rate limiting. This has been open since v1 and remains the most straightforward denial-of-service vector.

Affected endpoints: `/api/endorsement-request`, `/api/e4p-pledge-sign`, `/api/qgiv-webhook`, `/api/membership-invite`, `/api/events`, `/api/projects`

FIX — CLOUDFLARE WAF RATE LIMITING (FREE TIER)

```
# Cloudflare dashboard → Security → WAF → Rate Limiting Rules
# Rule 1: Write endpoints
Expression: (http.request.method eq "POST") and (starts_with(http.request.uri.path,
"/api/"))
Action: Block  |  Threshold: 10 requests per 60 seconds per IP

# Rule 2: Webhook (stricter — should only receive QGiv server IPs)
Expression: starts_with(http.request.uri.path, "/api/qgiv-webhook")
Action: Block  |  Threshold: 30 requests per 60 seconds per IP
```

OPEN FINDINGS — MEDIUM SEVERITY

MEDIUM M-1

No CSRF Protection (Origin Header Check) on Form Endpoints

Files: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A01:2021 – Broken Access Control

CORS has been correctly restricted to `https://techforpalestine.org`. However, CORS is a *browser-side* policy — it prevents cross-origin browser requests from reading responses but does not prevent the server from processing them. A server-side `Origin` header check is needed to actually reject cross-origin POST requests before they reach the Notion write logic.

FIX — VERIFY ORIGIN BEFORE PROCESSING

```
// Add at the top of both POST handlers, before reading body
const origin = request.headers.get("Origin");
if (origin !== "https://techforpalestine.org") {
  return new Response(JSON.stringify({ error: "Forbidden" }), {
    status: 403,
    headers: { "Content-Type": "application/json" },
  });
}
```

MEDIUM M-2

No Input Format or Length Validation on Form Endpoints

Files: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A03:2021 – Injection

Field presence is checked, but there is no validation of email format, URL format (website, campaign link, and LinkedIn fields accept any string including `javascript:` URIs), or string length limits. A malicious actor can submit fields of arbitrary length, potentially storing megabytes in Notion.

FIX — SERVER-SIDE VALIDATION

```
// endorsement-request.ts — add after the presence check
const emailRx = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRx.test(endorsementData.contactEmail)) { /* 400 */ }

try { new URL(endorsementData.organizationWebsite); }
catch { /* 400 — invalid URL */ }

const MAX = 2000;
if (endorsementData.campaignPurpose.length > MAX) { /* 400 */ }
// Apply similar checks to all free-text fields
```

MEDIUM M-4

Donor Email Addresses Still Logged in Webhook (PII in Server Logs)

File: src/pages/api/qgiv-webhook.ts:51, 53, 57, 76, 78, 82, 85

OWASP: A09:2021 – Security Logging Failures

Good progress: The bulk verbose logging (full payload dump, S3 signed URL objects) has been removed from `notionClient.ts` and `api.ts`. However, the webhook still logs donor email addresses in multiple places — both on success and failure paths:

```
// qgiv-webhook.ts — email in logs
console.log(`✅ Hub invite sent to ${email}`);          // line 51
console.error(`Hub invite failed for ${email}: ...`);    // line 53
console.warn(`...skipping invite for ${email}`);          // line 57
console.log(`✅ EmailOctopus contact added for ${email}`); // line 76
```

Email addresses are PII under GDPR/CCPA. Logging them to Cloudflare's log store without explicit data retention policies creates compliance and data leakage risk.

FIX

Replace email addresses in logs with a redacted form or a hashed reference: `email.split("@")[1]` (domain only) or `crypto.createHash("sha256").update(email).digest("hex").slice(0,8)` (anonymous correlation ID).

OPEN FINDINGS — LOW SEVERITY

LOW L-2

QGiv Payment Script Loaded Without Subresource Integrity Hash

File: src/components/MembershipPage.tsx **OWASP:** A08:2021 – Software & Data Integrity Failures

`https://secure.qgiv.com/resources/core/js/embed.js` is dynamically injected without an `integrity` attribute. A supply-chain compromise of QGiv's CDN could deliver malicious JavaScript in the payment context. **Fix:** Add `integrity="sha384-<hash>"` computed from a known-good version of the script.

LOW L-3

Notion Image Proxy Has No Authentication

File: cloudflare-worker/worker.js **OWASP:** A01:2021 – Broken Access Control

The proxy validates that decoded URLs point to Notion S3 domains (preventing open-relay abuse), but any actor can use it to cache and serve any Notion S3 URL they possess. Low severity given images are already public via Notion's CDN. No immediate action required, but consider adding an `X-Proxy-Token` check for defence in depth.

SECURITY PROGRESS OVERVIEW

Round	Critical	High	Medium	Low	Total Open
v1 (2026-03-31)	3	5	6	3	17
v2 (2026-04-02)	1 new	6	7	3	17
v3 (2026-04-06)	**0**	**2**	**5**	**3**	**10**

PRIORITISED REMEDIATION PLAN — V3

Priority	Action	ID	Effort
Immediate	Remove `payload` from the webhook fallthrough response — one-line fix	H-2	5 min
Day 1	Replace `'unsafe-inline'` in CSP with nonce or hash-based approach	R-2	2–4 hours
Day 1	Implement Cloudflare WAF rate limiting rules on POST and webhook endpoints	H-4	1 hour
Day 1	Add `Origin` header verification to `endorsement-request` and `e4p-pledge-sign`	M-1	30 min
Week 1	Proxy `api.ts` requests through a server-side route — remove `PUBLIC_SECRET_KEY` from browser bundle	R-1	3 hours
Week 1	Add email/URL format and length validation to both form POST endpoints	M-2	2 hours
Week 1	Replace email addresses in webhook logs with domain-only or hashed identifiers	M-4	30 min
Week 2	Add SRI `integrity` hash to QGiv embed script injection	L-2	30 min

GOOD PRACTICES CONFIRMED IN V3 CODEBASE

- **crypto.ts utility is well-implemented:** Uses `timingSafeEqual` from `node:crypto` with `TextEncoder` — correct and idiomatic for the Cloudflare runtime.

- **getEnv is clean:** Three-source resolution (CF runtime → import.meta.env → process.env) with no unsafe fallbacks. Simple and correct.
- **Webhook now uses header-based auth correctly:** `X-Webhook-Secret` header keeps the secret out of URLs and logs.
- **membership-invite validates and whitelists all fields** before proxying — no mass-assignment risk.
- **Middleware cache logic is correct:** Non-GET and API routes get `no-store`; only static GET page responses are cached.
- **CORS restricted to origin** on all write endpoints. OPTIONS handler on webhook also correct.
- **Security headers deployed** with a solid policy base — just the `'unsafe-inline'` needs hardening.
- **No more verbose debug logging** in notionClient.ts or the image proxy for S3 URLs and file objects.
- **Approval gating on signatories** still in place — new pledge submissions are not immediately public.